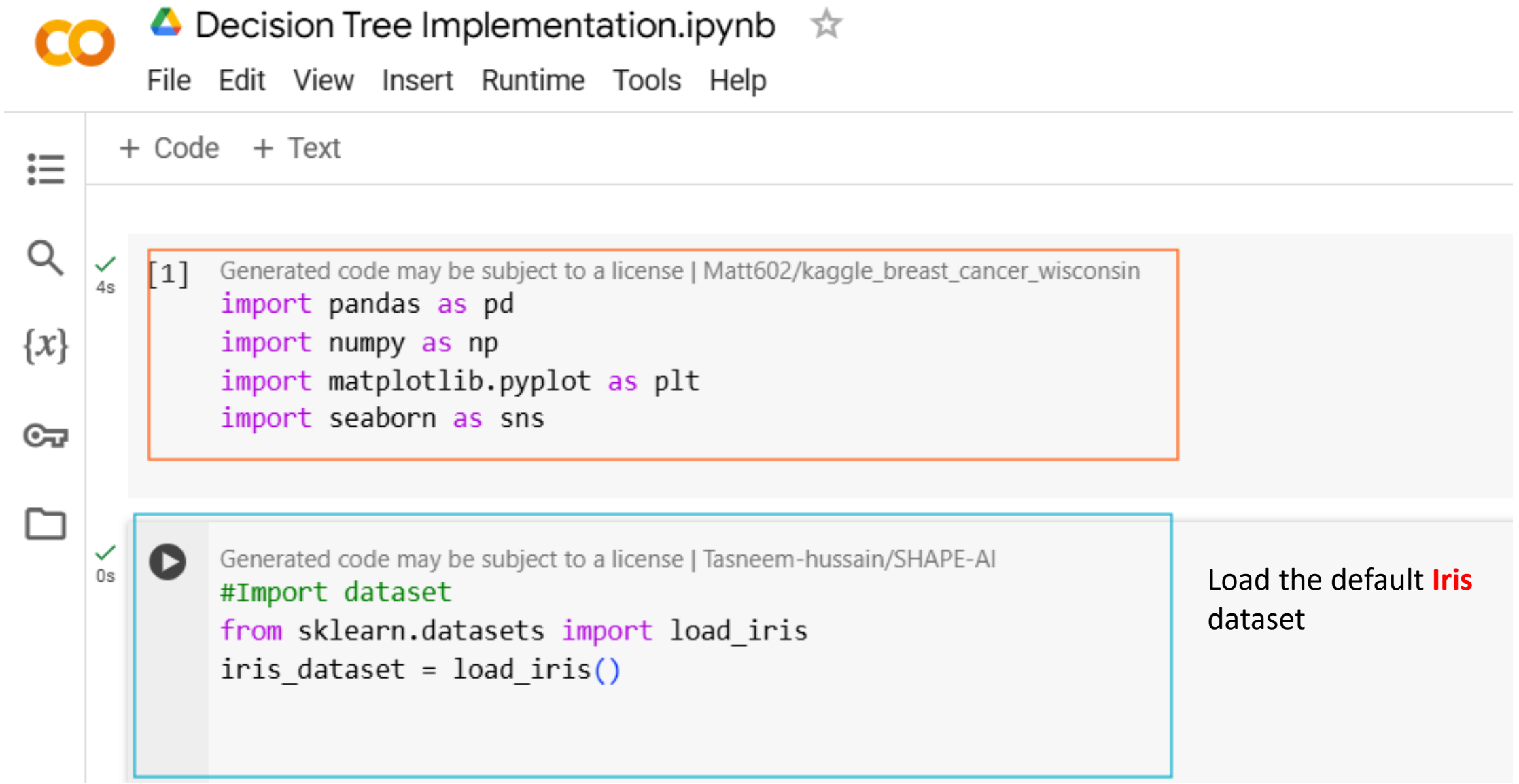
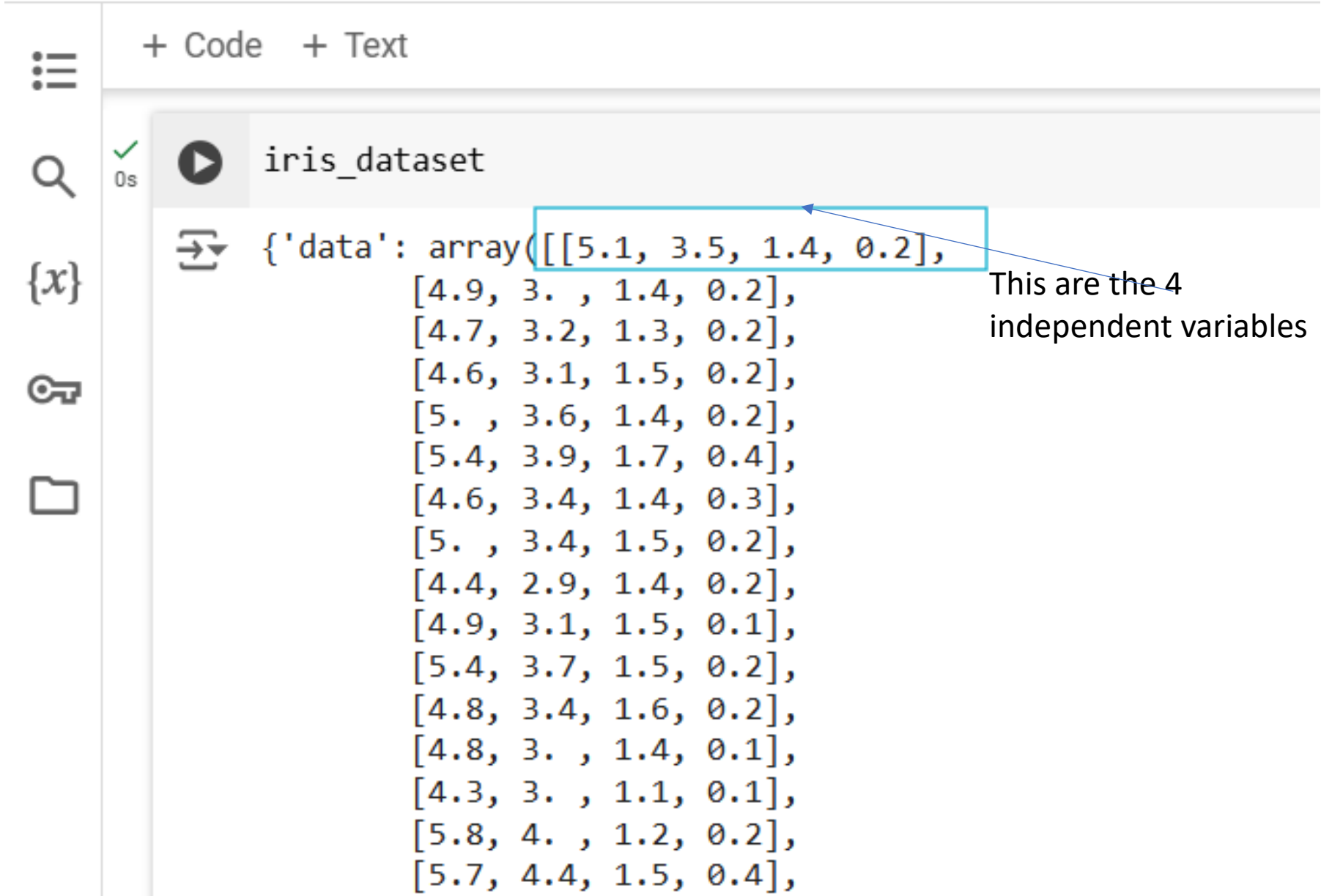
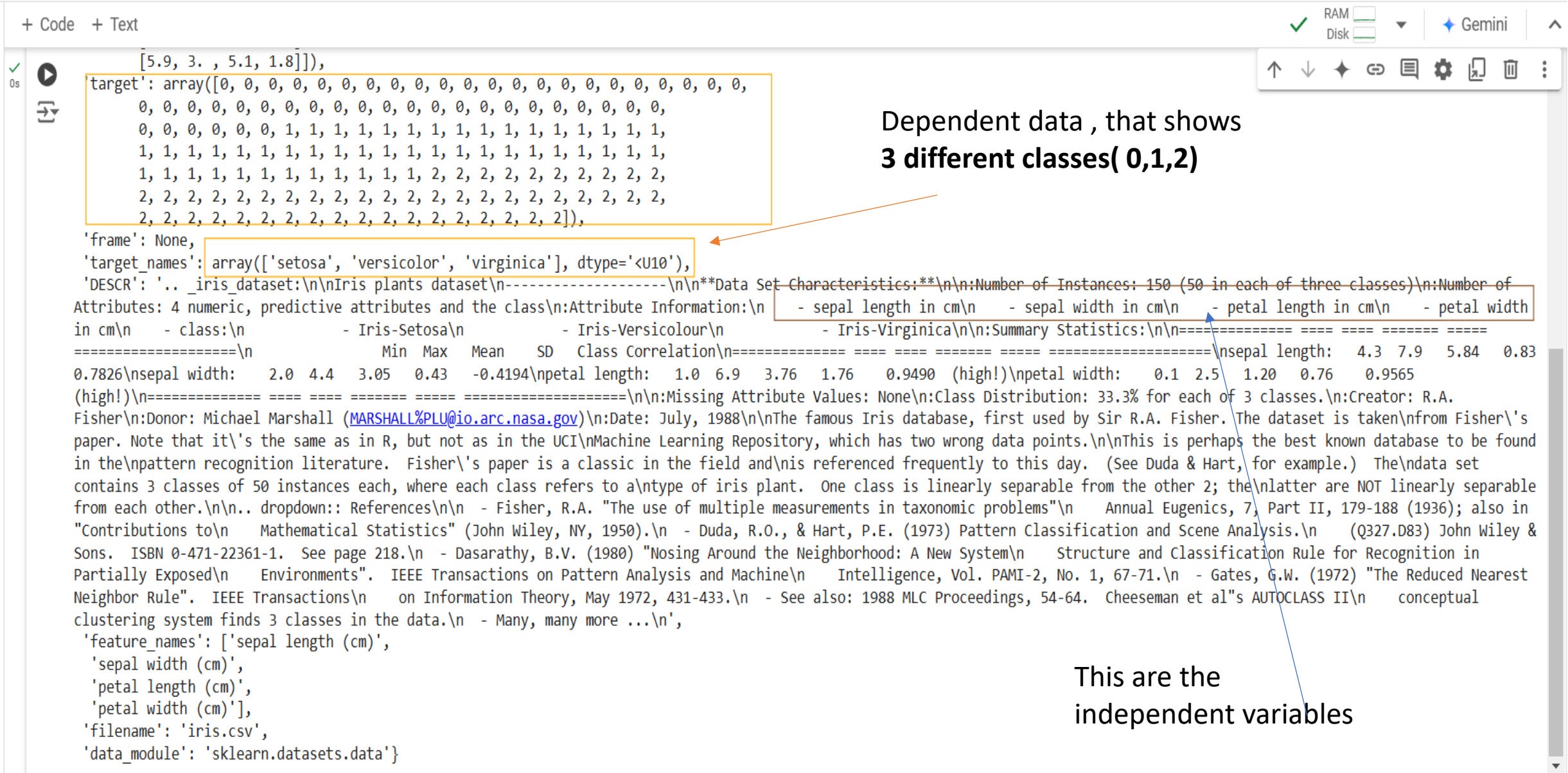

Decision Tree Implementation.ipynb
File Edit View Insert Runtime Tools Help
+ Code + Text
[1]
Generated code may be subject to a license | Matt602/kaggle_breast_cancer_wisconsin
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
4s
Generated code may be subject to a license | Tasneem-hussain/SHAPE-AI
#Import dataset
from sklearn.datasets import load_iris
iris_dataset = load_iris()
0s
Load the default Iris dataset


+ Code
+ Text
0s
iris_dataset
{'data': array([[5.1, 3.5, 1.4, 0.2],
[4.9, 3. , 1.4, 0.2],
[4.7, 3.2, 1.3, 0.2],
[4.6, 3.1, 1.5, 0.2],
[5. , 3.6, 1.4, 0.2],
[5.4, 3.9, 1.7, 0.4],
[4.6, 3.4, 1.4, 0.3],
[5. , 3.4, 1.5, 0.2],
[4.4, 2.9, 1.4, 0.2],
[4.9, 3.1, 1.5, 0.1],
[5.4, 3.7, 1.5, 0.2],
[4.8, 3.4, 1.6, 0.2],
[4.8, 3. , 1.4, 0.1],
[4.3, 3. , 1.1, 0.1],
[5.8, 4. , 1.2, 0.2],
[5.7, 4.4, 1.5, 0.4],
This are the 4
independent variables




```
        [5.9, 3. , 5.1, 1.8]]),
 'target': array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
        2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
        2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]),
 'frame': None,
 'target_names': array(['setosa', 'versicolor', 'virginica'], dtype='<U10'),
 'DESCR': '.. _iris_dataset:\n\nIris plants dataset\n--------------------\n\n**Data Set Characteristics:**\n\n:Number of Instances: 150 (50 in each of three classes)\n:Number of
Attributes: 4 numeric, predictive attributes and the class\n:Attribute Information:\n    - sepal length in cm\n    - sepal width in cm\n    - petal length in cm\n    - petal width
in cm\n    - class:\n            - Iris-Setosa\n            - Iris-Versicolour\n            - Iris-Virginica\n\n:Summary Statistics:\n\n============== ==== ==== ======= =====
====================\n                Min  Max   Mean    SD   Class Correlation\n============== ==== ==== ======= ===== ====================\nsepal length:   4.3  7.9   5.84   0.83
0.7826\nsepal width:    2.0  4.4   3.05   0.43   -0.4194\npetal length:   1.0  6.9   3.76   1.76    0.9490  (high!)\npetal width:    0.1  2.5   1.20   0.76    0.9565
(high!)\n============== ==== ==== ======= ===== ====================\n\n:Missing Attribute Values: None\n:Class Distribution: 33.3% for each of 3 classes.\n:Creator: R.A.
Fisher\n:Donor: Michael Marshall (MARSHALL%PLU@io.arc.nasa.gov)\n:Date: July, 1988\n\nThe famous Iris database, first used by Sir R.A. Fisher. The dataset is taken\nfrom Fisher\'s
paper. Note that it\'s the same as in R, but not as in the UCI\nMachine Learning Repository, which has two wrong data points.\n\nThis is perhaps the best known database to be found
in the\npattern recognition literature.  Fisher\'s paper is a classic in the field and\nis referenced frequently to this day.  (See Duda & Hart, for example.)  The\ndata set
contains 3 classes of 50 instances each, where each class refers to a\ntype of iris plant.  One class is linearly separable from the other 2; the\nlatter are NOT linearly separable
from each other.\n\n.. dropdown:: References\n\n  - Fisher, R.A. "The use of multiple measurements in taxonomic problems"\n    Annual Eugenics, 7, Part II, 179-188 (1936); also in
"Contributions to\n    Mathematical Statistics" (John Wiley, NY, 1950).\n  - Duda, R.O., & Hart, P.E. (1973) Pattern Classification and Scene Analysis.\n    (Q327.D83) John Wiley &
Sons.  ISBN 0-471-22361-1.  See page 218.\n  - Dasarathy, B.V. (1980) "Nosing Around the Neighborhood: A New System\n    Structure and Classification Rule for Recognition in
Partially Exposed\n    Environments".  IEEE Transactions on Pattern Analysis and Machine\n    Intelligence, Vol. PAMI-2, No. 1, 67-71.\n  - Gates, G.W. (1972) "The Reduced Nearest
Neighbor Rule".  IEEE Transactions\n    on Information Theory, May 1972, 431-433.\n  - See also: 1988 MLC Proceedings, 54-64.  Cheeseman et al"s AUTOCLASS II\n    conceptual
clustering system finds 3 classes in the data.\n  - Many, many more ...\n',
 'feature_names': ['sepal length (cm)',
  'sepal width (cm)',
  'petal length (cm)',
  'petal width (cm)'],
 'filename': 'iris.csv',
 'data_module': 'sklearn.datasets.data'}
```

Decision Tree Implementation.ipynb

File Edit View Insert Runtime Tools Help All changes saved

+ Code + Text

```
print(iris_dataset['DESCR'])
```

```
.. _iris_dataset:

Iris plants dataset
--------------------

**Data Set Characteristics:**

:Number of Instances: 150 (50 in each of three classes)
:Number of Attributes: 4 numeric, predictive attributes and the class
:Attribute Information:
    - sepal length in cm
    - sepal width in cm
    - petal length in cm
    - petal width in cm
    - class:
            - Iris-Setosa
            - Iris-Versicolour
            - Iris-Virginica

:Summary Statistics:

============== ==== ==== ======= ===== ====================
                Min  Max   Mean    SD   Class Correlation
============== ==== ==== ======= ===== ====================
sepal length:   4.3  7.9   5.84   0.83    0.7826
sepal width:    2.0  4.4   3.05   0.43   -0.4194
petal length:   1.0  6.9   3.76   1.76    0.9490  (high!)
petal width:    0.1  2.5   1.20   0.76    0.9565  (high!)
============== ==== ==== ======= ===== ====================

:Missing Attribute Values: None
```

Decision Tree Implementation.ipynb

File Edit View Insert Runtime Tools Help All changes saved

+ Code + Text

0s

```
print(iris_dataset['target'])
```

```
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2
 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 2 2]
```

Manually we are giving the column names

```
#Independent features
x=pd.DataFrame(iris_dataset['data'],columns=['sepal_length','sepal_width','petal_length','petal_width'])
x.head()
```

	sepal_length	sepal_width	petal_length	petal_width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

[12] Generated code may be subject to a license |

```
#dependent Feature
y=iris_dataset['target']
```

```
#Train test split
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
```

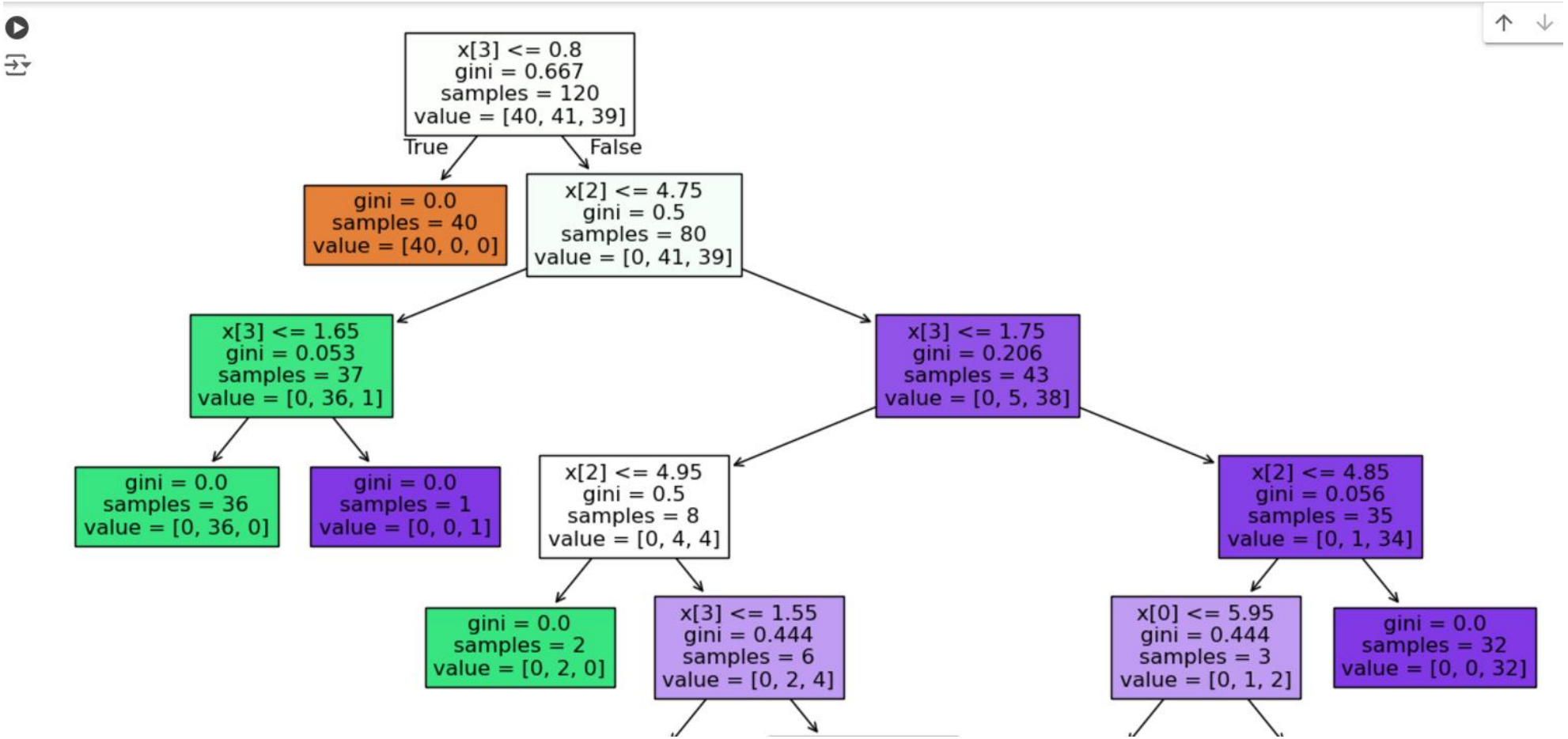
```
#apply decision tree classifier
from sklearn.tree import DecisionTreeClassifier
clf=DecisionTreeClassifier()
clf.fit(x_train,y_train)
```

```
DecisionTreeClassifier
DecisionTreeClassifier()
```

The **DecisionTreeClassifier** in scikit-learn is based on the **CART** algorithm.
CART uses **Gini Impurity** as its default criterion for classification tasks..

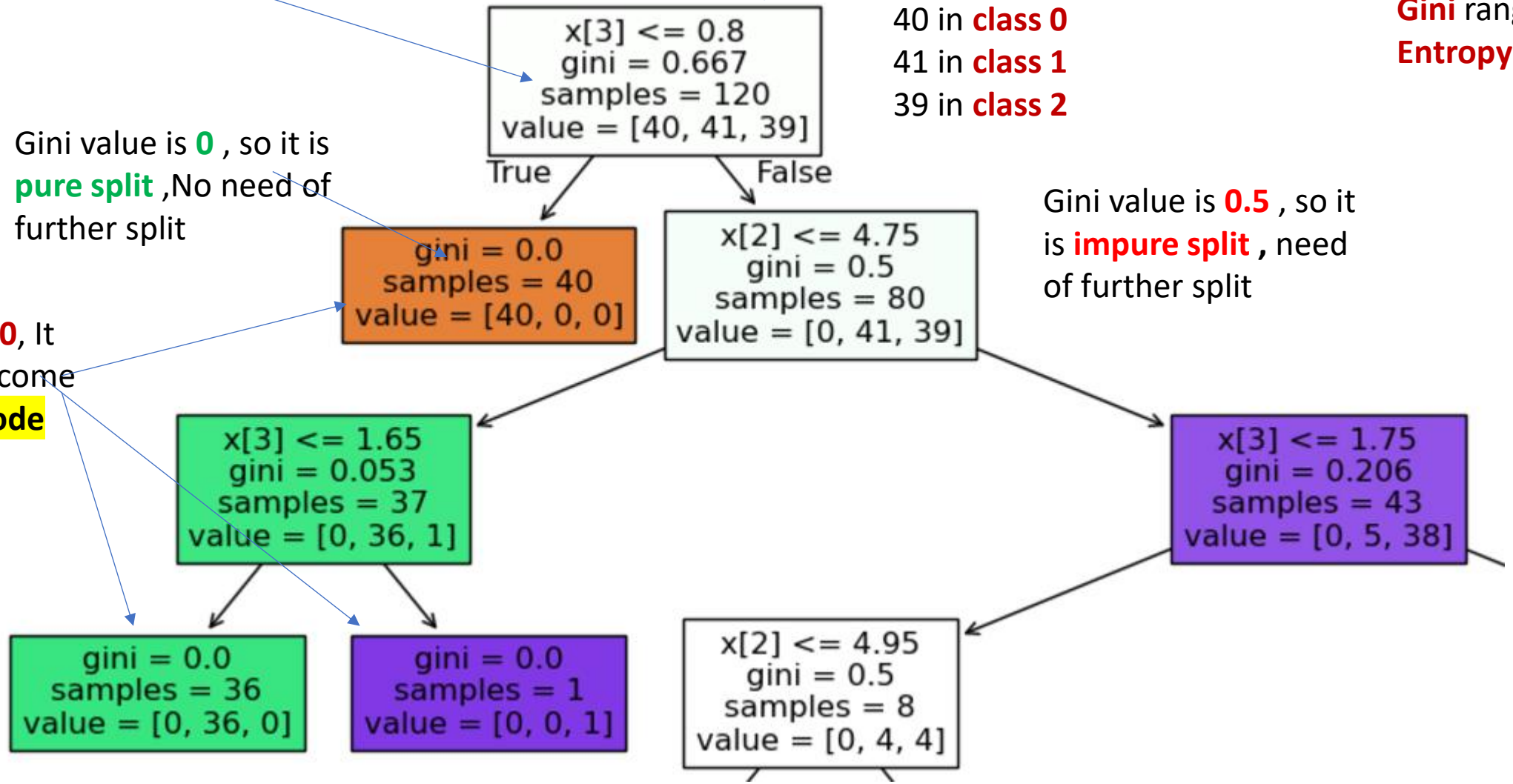
2s

```
#visualize the decison tree
from sklearn import tree
plt.figure(figsize=(15,10))
tree.plot_tree(clf,filled=True)
```



Decision is based on the **4th feature's** value is **less than or equal to 0.8** **(Threshold)**.

count of samples



40 in **class 0**
41 in **class 1**
39 in **class 2**

Gini range= 0 - 0.5
Entropy range = 0-1

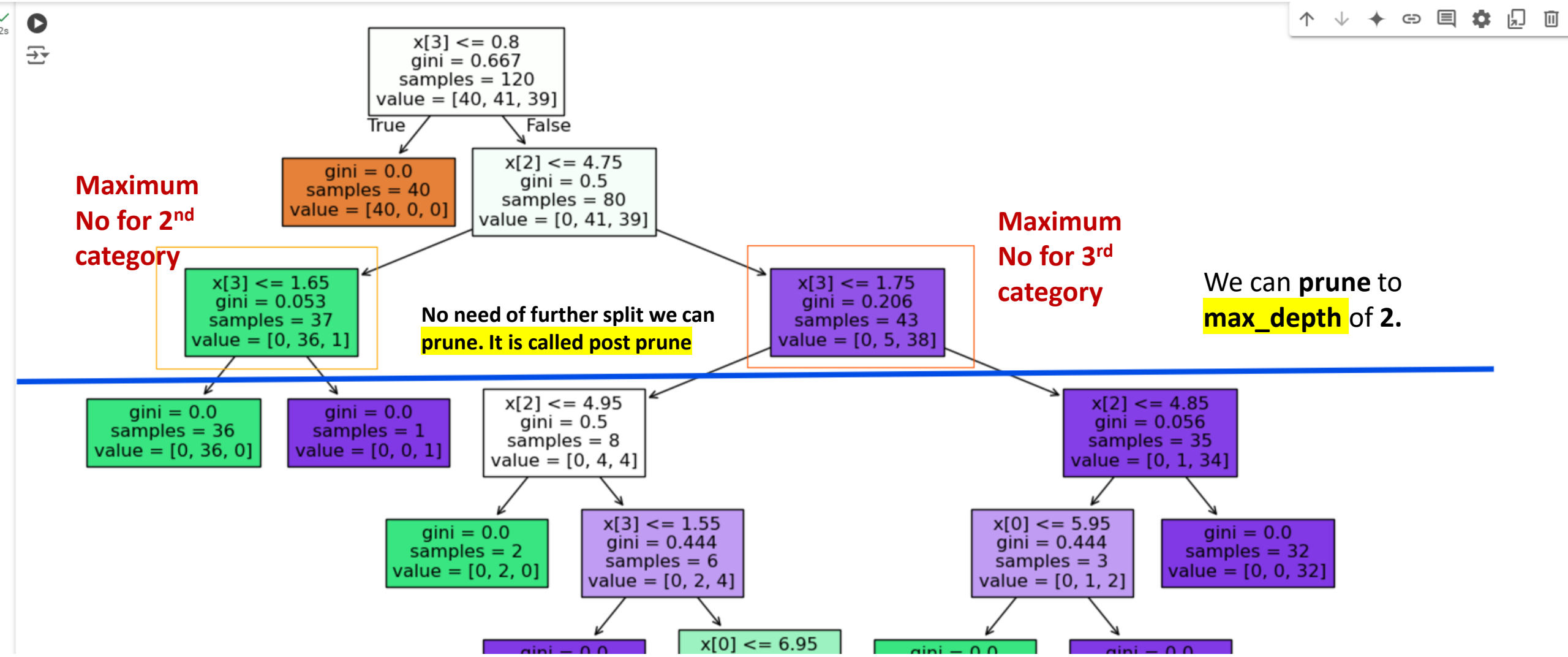
Gini value is **0** , so it is **pure split** ,No need of further split

Gini value is **0.5** , so it is **impure split** **,** need of further split

Gini is **0**, It will become **leaf Node**

We know that when we are constructing **decision tree for entire training data** , it will lead to **overfitting.**

So we can do either **post pruning or Pre pruning**



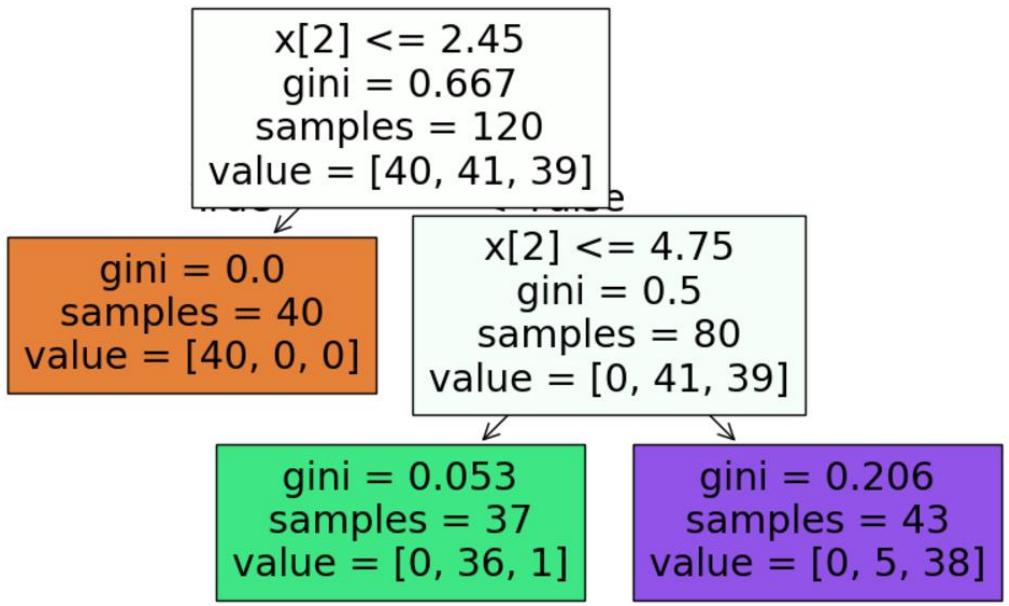
```
[17] #apply decision tree classifier
from sklearn.tree import DecisionTreeClassifier
clf=DecisionTreeClassifier(max_depth=2)
clf.fit(x_train,y_train)
```

```
DecisionTreeClassifier
DecisionTreeClassifier(max_depth=2)
```

Give **max_depth** parameter =2

Execute again

```
#visualize the decison tree
from sklearn import tree
plt.figure(figsize=(15,10))
tree.plot_tree(clf,filled=True)
```



```
[21] #prediction
     y_pred = clf.predict(x_test)
```

```
from sklearn.metrics import accuracy_score,confusion_matrix,classification_report
print(accuracy_score(y_test,y_pred))
print(confusion_matrix(y_test,y_pred))
print(classification_report(y_test,y_pred))
```

```
0.9666666666666667
[[10  0  0]
 [ 0  8  1]
 [ 0  0 11]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      0.89      0.94         9
           2       0.92      1.00      0.96        11

    accuracy                           0.97        30
   macro avg       0.97      0.96      0.97        30
weighted avg       0.97      0.97      0.97        30
```

Decision Tree Classifier prepruning and hyper parameter tuning

Preprunning Parameters are basically key value pair

```
[26] param={
        'criterion':['gini','entropy'],
        'splitter':['best','random'],
        'max_depth':[1,2,3,4,5],
        'max_features':['auto','sqrt','log2']
     }
```

splitter='best': Use when you need **high accuracy** and have **manageable dataset size** and feature count.
Splitter='random': Use when working with **very large datasets.**

```
param
```

```
{'criterion': ['gini', 'entropy'],
 'splitter': ['best', 'random'],
 'max_depth': [1, 2, 3, 4, 5],
 'max_features': ['auto', 'sqrt', 'log2']}
```

Automatically selects the number of features based on the task

Reduces the likelihood of overfitting

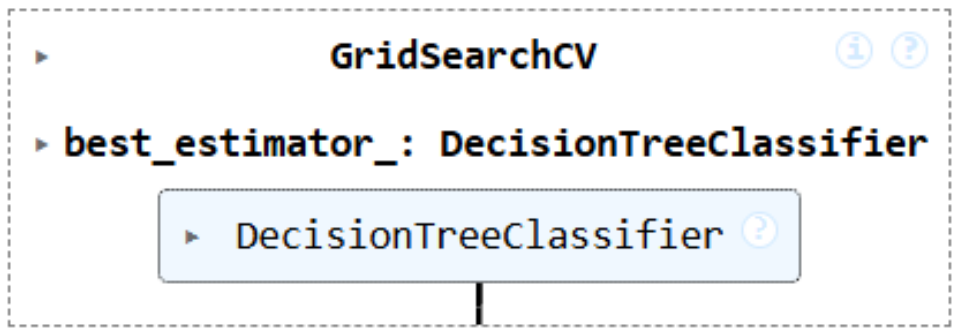
More restrictive, often used for **very high-dimensional data** to reduce computational cost.

```
[37] from sklearn.model_selection import GridSearchCV
     grid_search_model=GridSearchCV(estimator=clf,param_grid=param,cv=5,scoring='accuracy')
```

```
[41] import warnings
     warnings.filterwarnings('ignore')
```

This will remove the warning messages

```
grid_search_model.fit(x_train,y_train)
```



```
[43] grid_search_model.best_params_
```

```
{'criterion': 'entropy',
 'max_depth': 3,
 'max_features': 'sqrt',
 'splitter': 'best'}
```

```
[44] Generated code may be subject to a license | BHATNISAR/MARKETING-LEAD-CLASSIFIER
#prediction
y_pred = grid_search_model.predict(x_test)
```

```
Generated code may be subject to a license | HuangChingHan/Logistic-regression
#metrics
import sklearn.metrics as metrics
print(metrics.accuracy_score(y_test,y_pred))
print(metrics.confusion_matrix(y_test,y_pred))
print(metrics.classification_report(y_test,y_pred))
```

```
1.0
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

Diabetes Prediction using Decision tree regressor

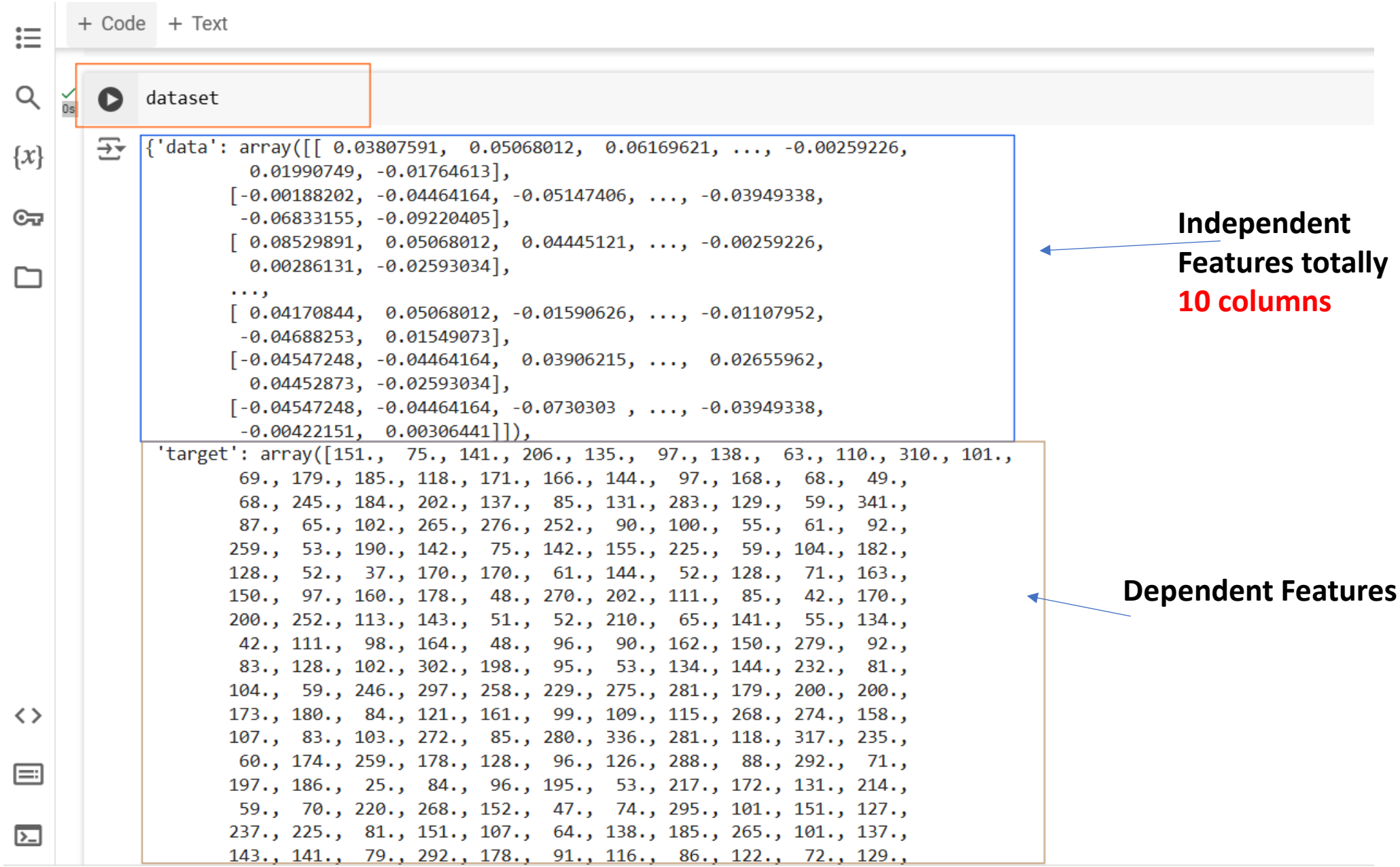
Diabetic prediction.ipynb

File Edit View Insert Runtime Tools Help All changes saved

+ Code + Text

[2] 8s
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

[4] 0s
```
#import dataset
from sklearn.datasets import load_diabetes
dataset = load_diabetes()
```


+ Code
+ Text
dataset
{'data': array([[0.03807591, 0.05068012, 0.06169621, ..., -0.00259226,
0.01990749, -0.01764613],
[-0.00188202, -0.04464164, -0.05147406, ..., -0.03949338,
-0.06833155, -0.09220405],
[0.08529891, 0.05068012, 0.04445121, ..., -0.00259226,
0.00286131, -0.02593034],
...,
[0.04170844, 0.05068012, -0.01590626, ..., -0.01107952,
-0.04688253, 0.01549073],
[-0.04547248, -0.04464164, 0.03906215, ..., 0.02655962,
0.04452873, -0.02593034],
[-0.04547248, -0.04464164, -0.0730303 , ..., -0.03949338,
-0.00422151, 0.00306441]]),
'target': array([151., 75., 141., 206., 135., 97., 138., 63., 110., 310., 101.,
69., 179., 185., 118., 171., 166., 144., 97., 168., 68., 49.,
68., 245., 184., 202., 137., 85., 131., 283., 129., 59., 341.,
87., 65., 102., 265., 276., 252., 90., 100., 55., 61., 92.,
259., 53., 190., 142., 75., 142., 155., 225., 59., 104., 182.,
128., 52., 37., 170., 170., 61., 144., 52., 128., 71., 163.,
150., 97., 160., 178., 48., 270., 202., 111., 85., 42., 170.,
200., 252., 113., 143., 51., 52., 210., 65., 141., 55., 134.,
42., 111., 98., 164., 48., 96., 90., 162., 150., 279., 92.,
83., 128., 102., 302., 198., 95., 53., 134., 144., 232., 81.,
104., 59., 246., 297., 258., 229., 275., 281., 179., 200., 200.,
173., 180., 84., 121., 161., 99., 109., 115., 268., 274., 158.,
107., 83., 103., 272., 85., 280., 336., 281., 118., 317., 235.,
60., 174., 259., 178., 128., 96., 126., 288., 88., 292., 71.,
197., 186., 25., 84., 96., 195., 53., 217., 172., 131., 214.,
59., 70., 220., 268., 152., 47., 74., 295., 101., 151., 127.,
237., 225., 81., 151., 107., 64., 138., 185., 265., 101., 137.,
143., 141., 79., 292., 178., 91., 116., 86., 122., 72., 129.,
Independent
Features totally
10 columns
Dependent Features

```
print(dataset['DESCR'])
```

```
.. _diabetes_dataset:

Diabetes dataset
----------------

Ten baseline variables, age, sex, body mass index, average blood
pressure, and six blood serum measurements were obtained for each of n =
442 diabetes patients, as well as the response of interest, a
quantitative measure of disease progression one year after baseline.

**Data Set Characteristics:**

:Number of Instances: 442

:Number of Attributes: First 10 columns are numeric predictive values

:Target: Column 11 is a quantitative measure of disease progression one year after baseline

:Attribute Information:
    - age     age in years
    - sex
    - bmi     body mass index
    - bp      average blood pressure
    - s1      tc, total serum cholesterol
    - s2      ldl, low-density lipoproteins
    - s3      hdl, high-density lipoproteins
    - s4      tch, total cholesterol / HDL
    - s5      ltg, possibly log of serum triglycerides level
    - s6      glu, blood sugar level
```

Dependent variable

Independent Feature variables

```
[7] #Dataframe for the dataset
    df_diabetes=pd.DataFrame(dataset.data,columns=dataset.feature_names)
```

```
df_diabetes.head()
```

	age	sex	bmi	bp	s1	s2	s3	s4	s5	s6
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019907	-0.017646
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493	-0.068332	-0.092204
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	-0.002592	0.002861	-0.025930
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	0.034309	0.022688	-0.009362
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592	-0.031988	-0.046641

Next steps: Generate code with df_diabetes | View recommended plots | New interactive sheet

```
[9]  #Independent and dependent features
     x= df_diabetes
     y=dataset.target
```

```
[10] #Train test split
     from sklearn.model_selection import train_test_split
     x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.3,random_state=10)
```

Generated code may be subject to a license |

```
#Build a decisiontree regressor
from sklearn.tree import DecisionTreeRegressor
regressor=DecisionTreeRegressor()
regressor.fit(x_train,y_train)
```

If we create a **Decision tree** Now ,definitely it will lead to **overfitting**, Because it will create a decision tree completely to the depth.

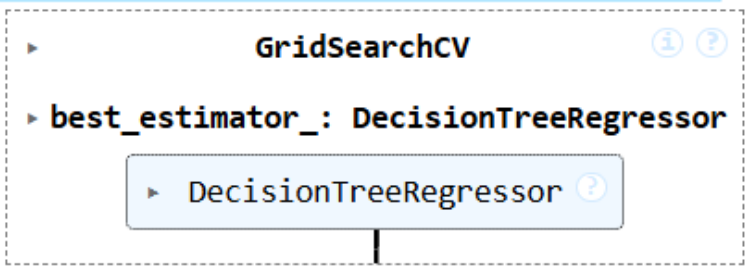
```
DecisionTreeRegressor
DecisionTreeRegressor()
```

Hyper parameter tuning

```
[13] param={
        'criterion':['squared_error','absolute_error','poisson'],
        'splitter':['best','random'],
        'max_depth':[3,5,7,9,10,12],
        'max_features':['auto','sqrt','log2'],
        }
```

```
[16] from sklearn.model_selection import GridSearchCV
     grid_search=GridSearchCV(estimator=regressor,param_grid=param,cv=5,scoring='neg_mean_squared_error')
```

```
import warnings
warnings.filterwarnings('ignore')
grid_search.fit(x_train,y_train)
```



```
[20] #predictions
     y_pred=grid_search.predict(x_test)
```

0s


```
#perfomance metrics
from sklearn.metrics import r2_score,mean_absolute_error,mean_squared_error
print(r2_score(y_test,y_pred))
print(mean_absolute_error(y_test,y_pred))
print(mean_squared_error(y_test,y_pred))
```

```
0.400621947721224 17
50.099476239172446
3792.667176400275
```

Build this model best on the **Best param** of **GridsearchCV**

To Visualize the Decision Tree Regressor

```
[25] new_model=DecisionTreeRegressor(criterion='poisson',max_depth=3,max_features='sqrt',splitter='best')
     new_model.fit(x_train,y_train)
```

```
DecisionTreeRegressor(criterion='poisson', max_depth=3, max_features='sqrt')
```

```
#visualize the decisiontree
from sklearn import tree
plt.figure(figsize=(10,7))
tree.plot_tree(new_model)
plt.show()
```